APS Shared Data Contract

Autonomous Pathogen Surveillance — Simulation Project

Purpose

This document defines the shared data formats used between components of the **Autonomous Pathogen Surveillance (APS)** simulation project.

Each subsystem may use its own internal implementation, algorithms, classes, and supporting files. However, any data passed between project components should follow the formats defined in this document unless the group agrees to modify the contract.

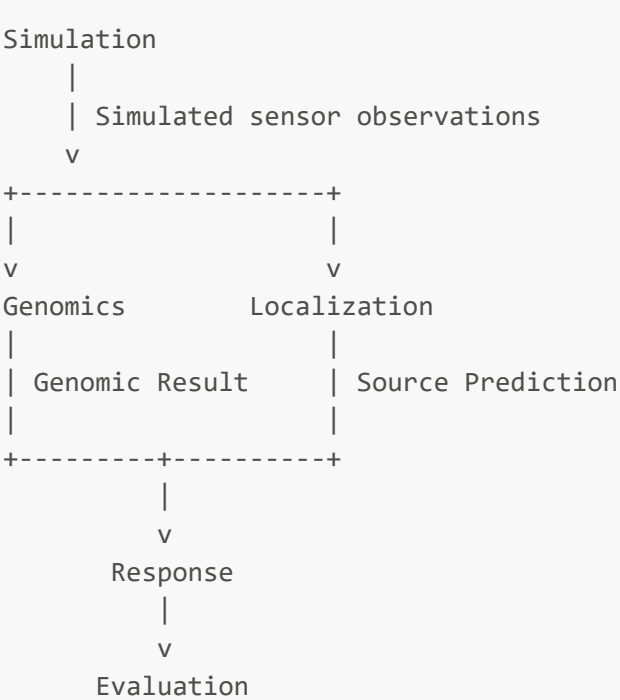
The purpose of this contract is to allow each team member to work independently while ensuring that all components can later be integrated into the complete APS pipeline.

> **Project Scope:** APS is a simulation-only undergraduate artificial intelligence project. It is not intended for clinical diagnosis, real-world pathogen surveillance, or medical decision-making.

---

1. APS Data Flow

The general flow of information through APS is:



Each component is responsible for producing predictable output that can be consumed by the next component.

---

2. General Data Conventions

Identifiers

All shared identifiers should use lowercase `snake_case` formatting.

Examples:

```
room_203
hallway_b
sensor_03
pathogen_001
sample_001
run_001
```

The same object must use the same identifier throughout the project.

For example:

```
Preferred:
room_203

Avoid mixing:
Room 203
room203
R203
room_203
```

Simulation Time

The initial APS simulation should use integer simulation steps:

```
0
1
2
3
...
```

The field name will be:

```
simulation_step
```

Real-world timestamps may be added later if required.

Boolean Values

Boolean values should be represented as:

```
true
false
```

Confidence and Probability Values

Confidence scores and probabilities should use decimal values from:

```
0.0 to 1.0
```

Example:

```
0.82 = 82% confidence
```

Missing Values

For JSON:

```
null
```

For CSV files, leave the field empty.

Avoid arbitrary missing-value representations such as:

```
N/A
none
unknown
-999
```

unless a specific component requires them.

---

3. Simulation Output

Primary Owner: Simulation & Sensor Modeling

The simulation generates the synthetic environment, outbreak behavior, and sensor observations used by the rest of APS.

Primary output:

```
Data/Synthetic/sensor_readings.csv
```

Required Format

```
run_id,simulation_step,sensor_id,node_id,pathogen_id,detected,confidence
run_001,10,sensor_03,room_203,pathogen_001,true,0.94
run_001,15,sensor_02,hallway_b,pathogen_001,true,0.81
run_001,20,sensor_05,room_204,pathogen_001,false,0.12
```

Required Fields

Field	Type	Description
`run_id`	string	Unique simulation/experiment identifier
`simulation_step`	integer	Current time step in the simulation
`sensor_id`	string	Identifier of the sensor producing the observation
`node_id`	string	Facility node where the sensor is located
`pathogen_id`	string	Identifier of the simulated pathogen
`detected`	boolean	Whether the sensor detected the pathogen
`confidence`	float	Simulated sensor confidence from `0.0` to `1.0`

---

4. Hidden Simulation Ground Truth

The simulator must also preserve the correct answer for later evaluation.

Example:

```
{
  "run_id": "run_001",
  "true_source_node": "room_203",
  "pathogen_id": "pathogen_001"
}
```

The ground-truth information should be stored separately from normal sensor observations.

Critical Rule

The localization algorithm **must not receive**:

```
true_source_node
```

That value is reserved for evaluation.

Otherwise, the model would receive the answer it is supposed to predict, creating **target leakage** and invalidating the experiment.

---

5. Genomics Input and Output

Primary Owner: Genomics & Mutation Analysis

The genomics component receives a reference genome and a simulated or observed variant genome.

Typical reference genome files will use FASTA format.

Example:

```
Data/ReferenceGenomes/reference.fasta
```

The genomics pipeline conceptually performs:

```
Reference Genome
       +
Variant Genome
       |
       v
Sequence Comparison
       |
       v
Detected Mutations
       |
       v
Genomic Risk / Anomaly Result
```

Required Genomics Output

```
{
  "sample_id": "sample_001",
  "pathogen_id": "pathogen_001",
  "reference_accession": "NC_045512.2",
  "mutation_count": 2,
  "mutations": [
    {
      "position": 23063,
```

```
        "reference_base": "A",
        "observed_base": "T",
        "mutation_type": "SNV"
      },
      {
        "position": 23403,
        "reference_base": "A",
        "observed_base": "G",
        "mutation_type": "SNV"
      }
    ],
    "risk_score": 0.67,
    "risk_label": "moderate"
  }
```

Required Fields

Field	Type	Description
`sample_id`	string	Unique genomic sample identifier
`pathogen_id`	string	Pathogen associated with the sample
`reference_accession`	string	Reference genome accession number
`mutation_count`	integer	Number of detected mutations
`mutations`	array	Individual mutation records
`risk_score`	float	Experimental anomaly/risk score
`risk_label`	string	Human-readable risk category

Initial risk labels:

```
low
moderate
high
```

These labels represent **experimental or simulated genomic anomaly risk**.

They should not be presented as clinically validated predictions of virulence, severity, or patient outcome.

---

6. Localization Input and Output

Primary Owner: Graph AI & Outbreak Localization

The localization component estimates where the simulated outbreak most likely originated.

It may use:

- facility graph structure
- sensor locations
- sensor detections
- simulation steps
- detection confidence
- modeled spread behavior

It must **not** access hidden ground truth.

Required Localization Output

```
{
  "run_id": "run_001",
  "predicted_source": "room_203",
  "confidence": 0.72,
  "top_candidates": [
    {
      "node_id": "room_203",
      "score": 0.72
    },
    {
      "node_id": "hallway_b",
      "score": 0.18
    },
    {
      "node_id": "room_204",
      "score": 0.07
    }
  ],
  "method": "graph_source_localization"
}
```

Required Fields

Field	Type	Description
`run_id`	string	Simulation run being analyzed
`predicted_source`	string	Most likely outbreak-source node
`confidence`	float	Confidence in the top prediction
`top_candidates`	array	Ranked source-node candidates
`method`	string	Localization method used

The value of:

```
predicted_source
```

must correspond to a valid facility `node_id`.

Preferred terminology:

```
source_node
predicted_source
probable_origin
outbreak_origin
```

Avoid using **Patient Zero**, because APS predicts a location rather than identifying an individual person.

---

7. Combined APS Result

Primary Owner: Data Pipeline / Response

The results of the genomic and localization components are combined into a standardized APS event.

Example:

```
{
  "run_id": "run_001",
  "pathogen_id": "pathogen_001",

  "genomics": {
    "mutation_count": 2,
    "risk_score": 0.67,
    "risk_label": "moderate"
  },

  "localization": {
    "predicted_source": "room_203",
    "confidence": 0.72
  },

  "response": {
    "recommendations": [
      "Increase simulated sampling near room_203",
      "Monitor adjacent facility nodes",
      "Inspect connected environmental pathways"
    ]
  }
}
```

The response component should consume the structured outputs produced by APS.

It should not directly access hidden simulation ground truth.

---

8. Evaluation Record

Primary Owner: Integration & Evaluation

Each completed simulation should produce an evaluation record comparing the model prediction against the simulator's known ground truth.

Example:

```
run_id,true_source,predicted_source,top1_correct,graph_distance_error,mutation_cou
nt,mutations_correct
run_001,room_203,room_203,true,0,2,true
```

Required Initial Fields

Field	Type	Description
`run_id`	string	Simulation run
`true_source`	string	Hidden source selected by simulator
`predicted_source`	string	Source predicted by localization
`top1_correct`	boolean	Whether the first prediction was correct
`graph_distance_error`	integer	Graph distance between true and predicted source
`mutation_count`	integer	Number of mutations in the scenario
`mutations_correct`	boolean	Whether genomic mutations were correctly identified

Additional evaluation metrics may be added as the project develops.

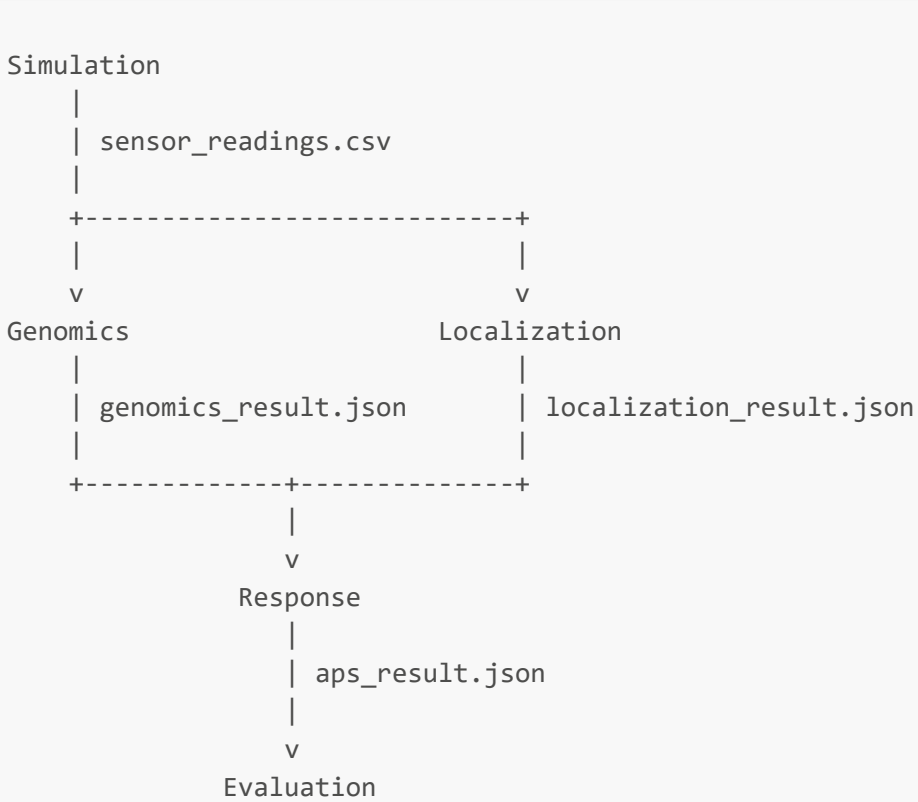
Possible future metrics include:

```
Top-3 source accuracy
Detection lead time
False-positive performance
False-negative performance
Missing-sensor robustness
Mean graph-distance error
Mutation detection precision
Mutation detection recall
```

---

9. Module Interface Summary

The expected information exchange between project components is:



The exact filenames may change during development, but the **field names and structures defined in this contract should remain consistent** unless the team agrees to modify them.

---

10. Interface Change Rule

Each team member is free to change the internal implementation of their assigned component.

For example, a team member may change:

- algorithms
- Python classes
- helper functions
- model architecture
- internal file structure
- implementation details

However, shared interfaces should not be changed independently.

For example, do not change:

```
node_id
```

into:

```
room
```

or:

```
location
```

or:

```
facility_node
```

without updating this contract and informing the team.

Likewise, do not change:

```
predicted_source
```

into another field name without coordinating with the components that consume the localization result.

---

11. Team Integration Rule

Before a component is considered ready for integration, its owner should verify that:

1. Required fields are present.
2. Field names match this contract.
3. Data types match this contract.
4. Identifiers use the agreed naming convention.
5. Missing values are handled consistently.
6. Hidden ground truth has not leaked into AI inputs.
7. The component produces output that another module can consume without manual rewriting.

The goal is for each subsystem to function independently while remaining compatible with the complete APS pipeline.

---

12. Current Contract Status

This document defines the **initial APS MVP data contract**.

It is expected to evolve as the project develops.

Changes should be made deliberately and documented through Git.

Recommended commit format:

```
docs: update APS shared data contract
```

The shared data contract should remain the authoritative reference whenever there is uncertainty about how information is exchanged between APS components.